

When you click an app, what exactly starts running inside the computer?
Is a program still a program when it is running?



Process

Program in action



Background



- Early computers ran **only one program at a time**
- The program had **full control of all system resources**
- Modern systems run **multiple programs concurrently**
- This led to the concept of a **process — a program in execution**

Basics



- A **process** is a program in execution.
- It is an active entity that uses CPU, memory, and I/O resources.
- Every running application in a system is a process.
- A process is an instance of a program currently being executed by the CPU.

Program vs Process

Program	Process
Passive (stored on disk)	Active (in execution)
No execution state	Has execution states
Single copy	Multiple processes possible

Components of a Process



Program Code (Text Section)



Program Counter (PC)



CPU Registers



Stack – function calls & local variables



Heap – dynamic memory allocation



Process State

Example with a code

- Program Code (Text Section):**

Stores program instructions

- Program Counter (PC):** Points to next instruction

- CPU Registers:** Perform fast calculations

- Stack:** Stores local variables and function calls

- Heap:** Stores dynamically allocated memory

- Process State:** Running → Waiting → Terminated

```
#include <stdio.h>
#include <stdlib.h>

/* Program Code (Text Section):
   All instructions of this program are stored here */
int add(int a, int b) {
    int sum = a + b;    // Stack + CPU registers used for calculation
    return sum;         // PC returns to main()
}

int main() {
    int x = 5, y = 3;    // Stack: local variables

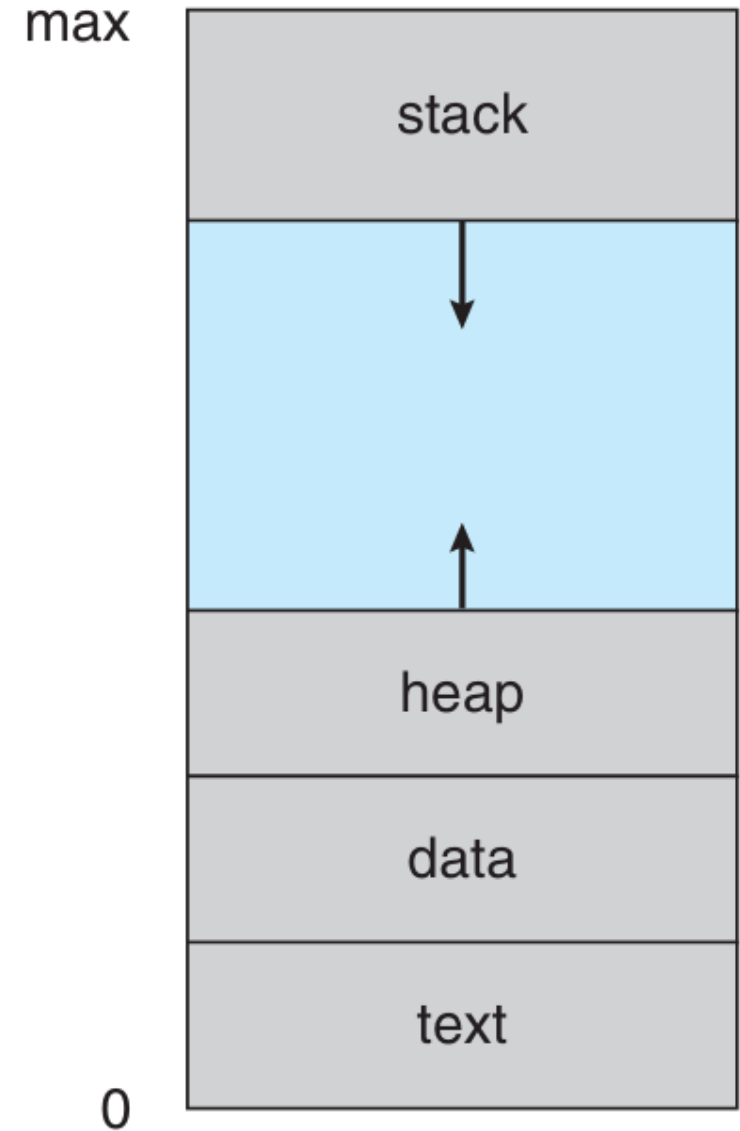
    int *p = malloc(sizeof(int)); // Heap: dynamic memory
    *p = add(x, y);        // PC jumps to add() and comes back

    printf("%d\n", *p);    // Process may wait for I/O

    free(p);              // Heap memory released
    return 0;             // Process terminated
}
```

Process in Memory

- **Text Section:** Stores the compiled program instructions (read-only).
- **Data Section:** Holds global and static variables for the entire execution.
- **Heap:** Used for dynamic memory allocation and grows upward.
- **Free Space:** Unused memory available between heap and stack.
- **Stack:** Stores function calls and local variables, growing downward.



Opposite Growth



- Heap grows upward because dynamic memory is allocated progressively from lower addresses as programs request more space.
- Stack grows downward because function calls are pushed from higher addresses toward lower ones.
- This opposite growth lets both share the same memory region efficiently.
- It maximizes memory usage and reduces wasted space between them.

Why stack at top?



- Function calls are unpredictable
- Recursion, nested calls → sudden growth
- Placing stack at top gives it maximum expansion room
- Heap grows gradually; stack may grow rapidly
- Separation delays stack–heap collision

Process States

New – process is being created

Ready – waiting for CPU

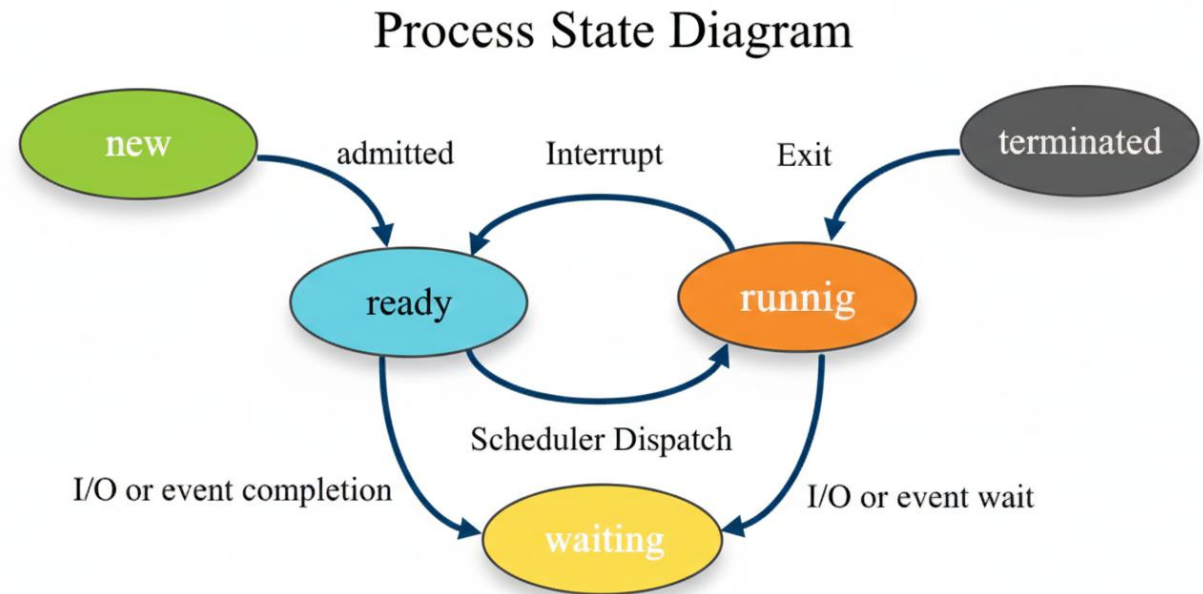
Running – executing on CPU

Waiting (Blocked) – waiting for I/O or event

Terminated – execution completed

Process State Diagram

- New → Ready → Running
- Running → Waiting (for I/O)
- Waiting → Ready
- Running → Terminated



How does the OS remember
who's running, who's
waiting, and who's done?

Process Control Block (PCB)



Process ID (PID)



Process State



Program Counter



CPU Registers



Memory
Management Info



Scheduling
Information

PCB

- **Process ID (PID):** Unique number that identifies each process in the system.
- **Process State:** Current status of the process (new, ready, running, waiting, terminated).
- **Program Counter:** Address of the next instruction to be executed.
- **CPU Registers:** Stored values of processor registers for context switching.
- **Memory Management Info:** Details about the process's memory (base/limit registers, page tables, etc.).
- **Scheduling Information:** Data used by the scheduler (priority, queue pointers, CPU time, etc.).

Role of PCB



Used by OS to manage processes



Helps in **context switching**



Ensures correct execution after interruption



PCB acts as an identity card of a process.

Context Switching



Occurs when CPU switches from one process to another



Current process state is saved in PCB



Next process state is loaded from PCB



Necessary for multitasking

Types of Processes



User Process – created by users



System Process – created by OS



Foreground Process



Background Process

Importance of Processes



Enables multitasking



Improves CPU
utilization



Allows concurrent
execution



Essential for modern
operating systems

Threads

- A traditional process executes only **one thread** (one task at a time)
- Multithreading allows a process to run **multiple tasks simultaneously**
- Useful in **multicore systems** for parallel execution
- PCB is extended to store **thread-related information**
- **Process** = Program in execution
- **Thread** = Smallest unit of execution inside a process

Process

Pros of Processes

- Strong isolation (one crash doesn't affect others)
- Better security
- Suitable for independent tasks

Cons of Processes

- Higher memory usage
- Slower creation and context switching
- Communication overhead (IPC required)

Threads

Pros of Threads

- Lightweight and faster to create
- Efficient data sharing
- Good for parallel execution on multicore systems

Cons of Threads

- Shared data can cause conflicts
- Debugging can be more complex
- One faulty thread may affect the entire program

Comparison

Aspect	Process	Thread
Memory	Separate memory space	Shared memory within same process
Communication	Requires IPC	Direct communication (shared data)
Creation Cost	High	Low
Context Switching	Slower	Faster
Isolation	Strong	Weak

Process Scheduling



Selects a ready process to run on the CPU core



Only **one process per core** can run at a time



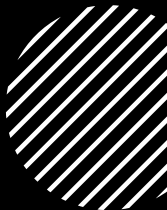
If processes $>$ cores $\rightarrow$ remaining processes wait



Degree of multiprogramming = number of processes in memory



Goals of Scheduling



Multiprogramming: Keep CPU busy at all times



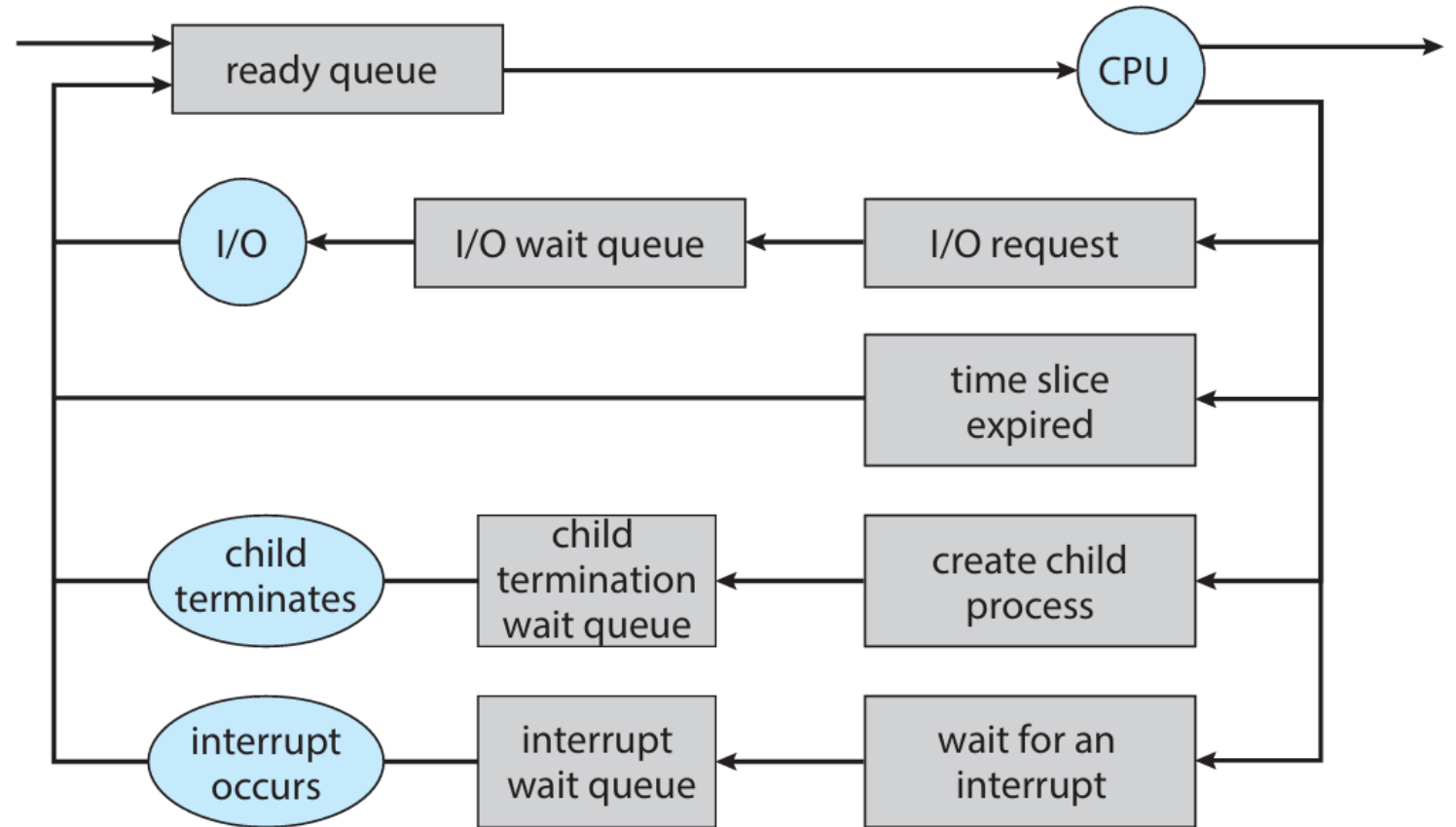
Time-sharing: Switch frequently for user interaction

Process Types

I/O-bound:
Spends more time
on I/O than
computation

CPU-bound:
Spends more time
on computation
than I/O

Scheduling Queues Illustration



Scheduling Queues

-
- New processes enter the **Ready Queue**
 - Ready queue stores processes waiting for CPU
 - Implemented as a linked list of PCBs
 - CPU Scheduler selects process from ready queue

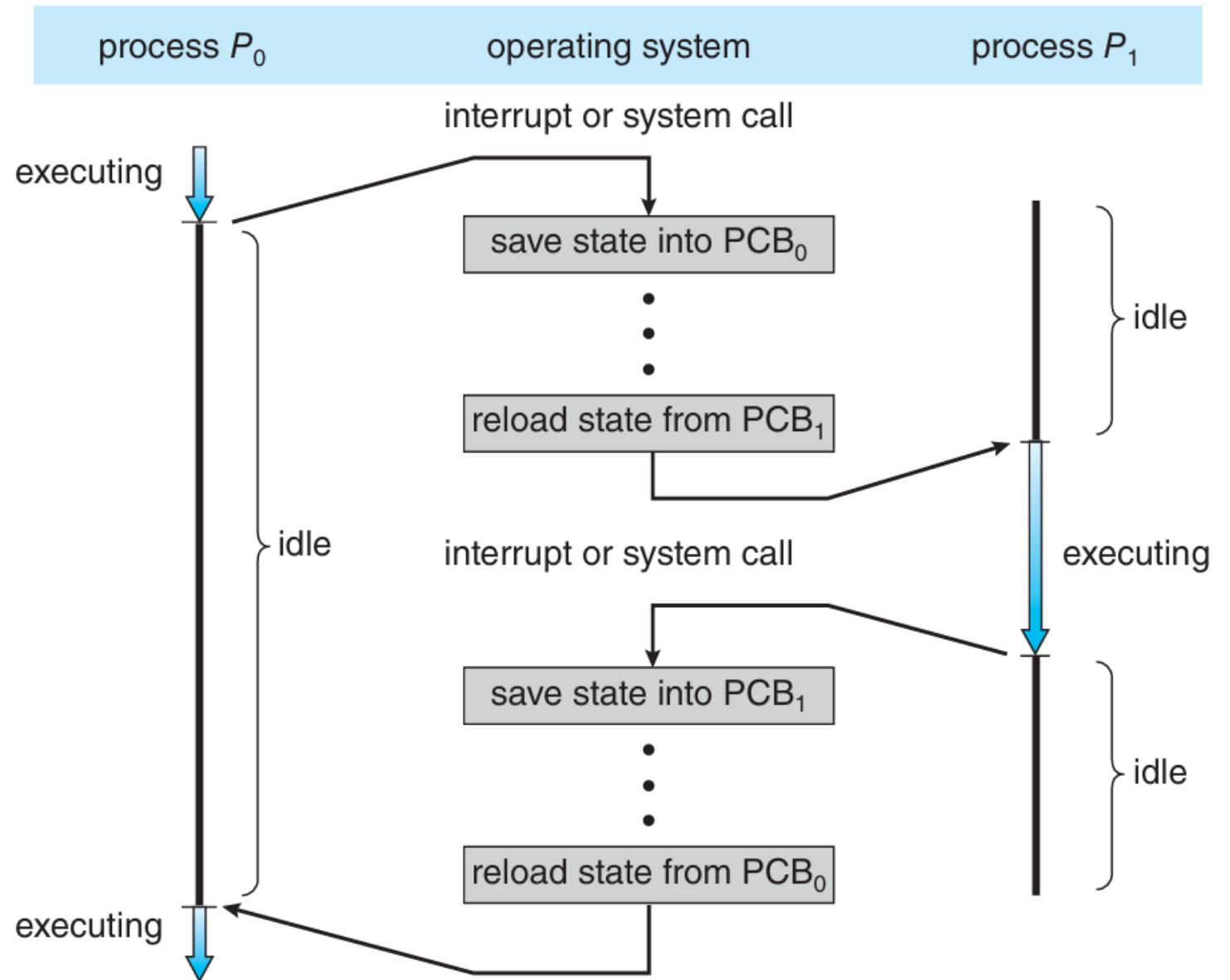
Wait Queues

-
- Issues an **I/O request** (disk, device, network)
 - Waits for **child process to terminate**
 - Waits for an **event or resource** to become available
 - Returns to Ready Queue after event completion

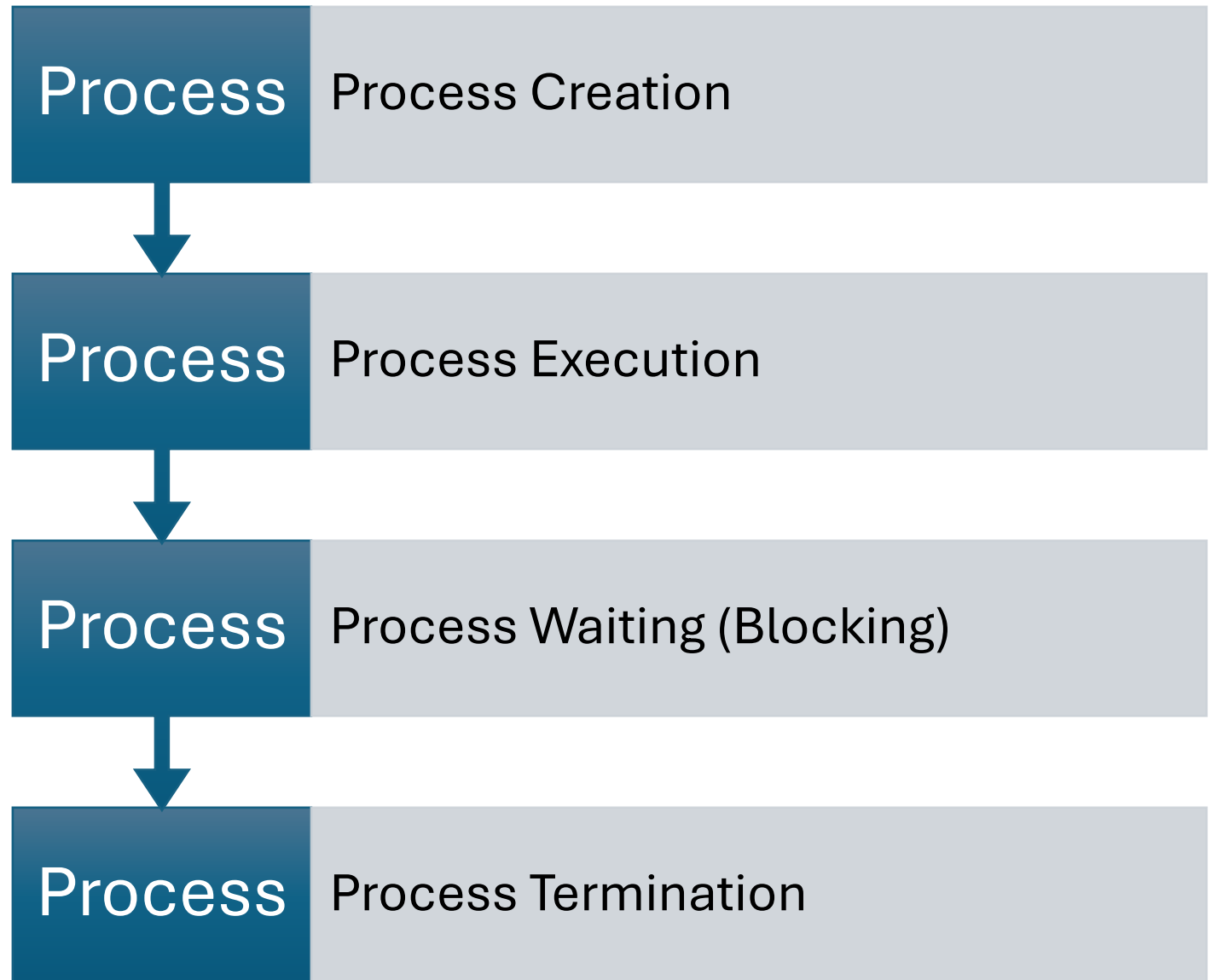
CPU Scheduler

-
- Processes move between **ready queue** and **wait queues** during execution
 - Scheduler selects a process from the **ready queue**
 - I/O-bound processes use CPU for short bursts
 - CPU-bound processes run longer but can be preempted

Context Switching



Operations on Process



Process Creation



New process created using system calls (e.g., `fork()`)



OS assigns a unique PID

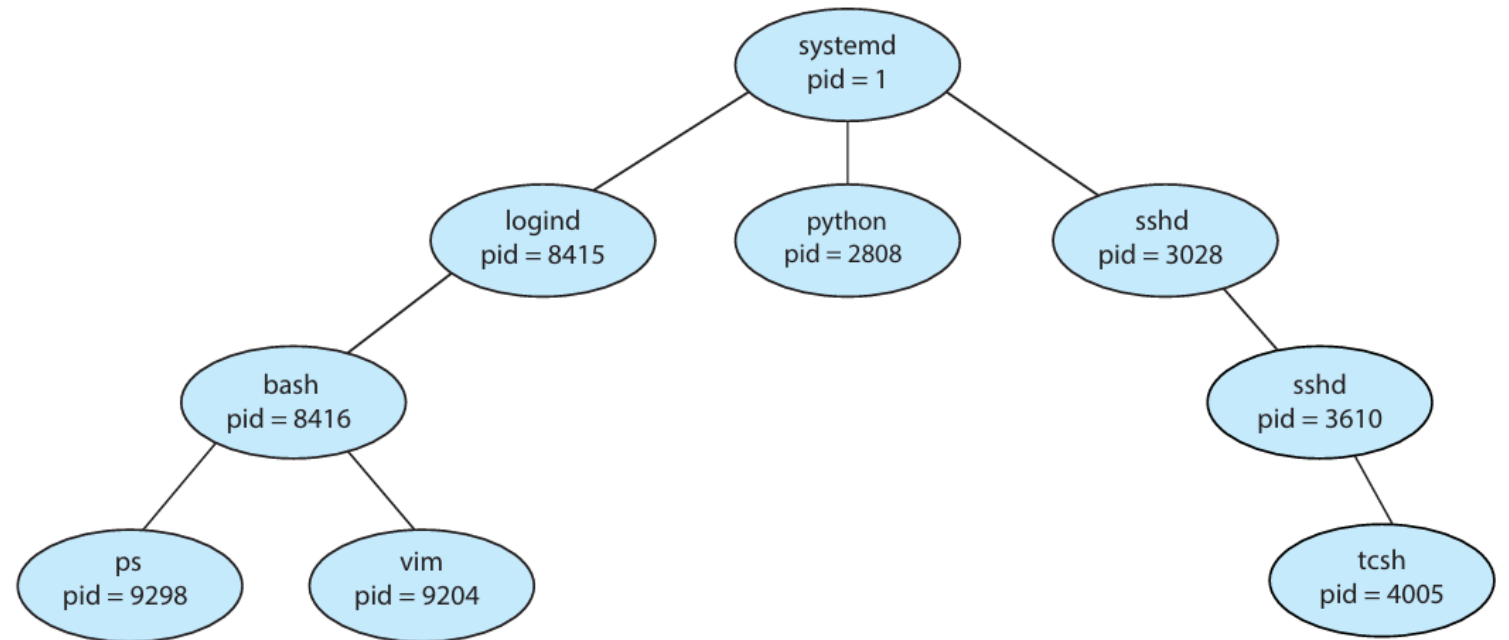


PCB is created and initialized



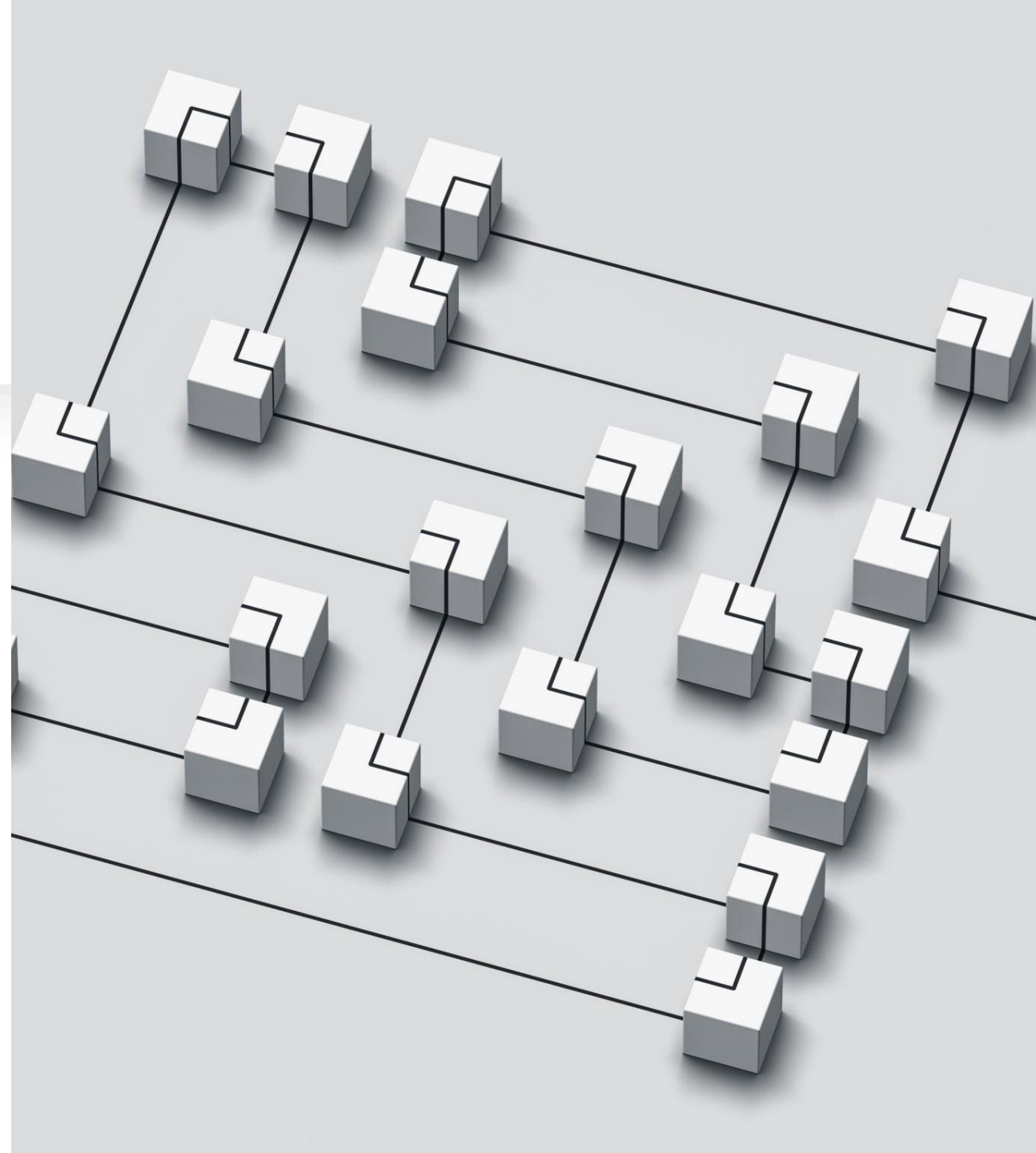
Process enters Ready state

Linux OS Process Tree



systemd and Process Hierarchy (Linux)

- **systemd (PID 1)** is the first user process started at boot
- It acts as the **root parent** of all other processes
- Creates and manages system services (ssh, print, web, etc.)
- All user processes are children (directly or indirectly) of systemd



Example Process Tree

- systemd → starts logind and sshd
- logind manages users logged in locally
- User runs bash shell → creates ps, vim, etc.
- sshd manages remote users via SSH





Process Execution



Scheduler selects process
from Ready Queue

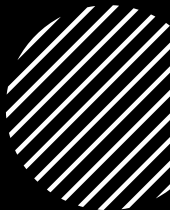
Dispatcher assigns CPU

Process moves to Running
state

May be preempted (time
slice/interrupt)



Process Waiting (Blocking)



Process requests I/O or waits for event



Moves from Running → Waiting state



CPU allocated to another process



Returns to Ready after event completion

Process Termination

Process finishes execution
(`exit()`)

May return status to parent
process

OS releases allocated
resources

PCB is removed from system

Process Transitions (Under the hood)

```
</> C

#include <stdio.h>
#include <unistd.h>

int main() {

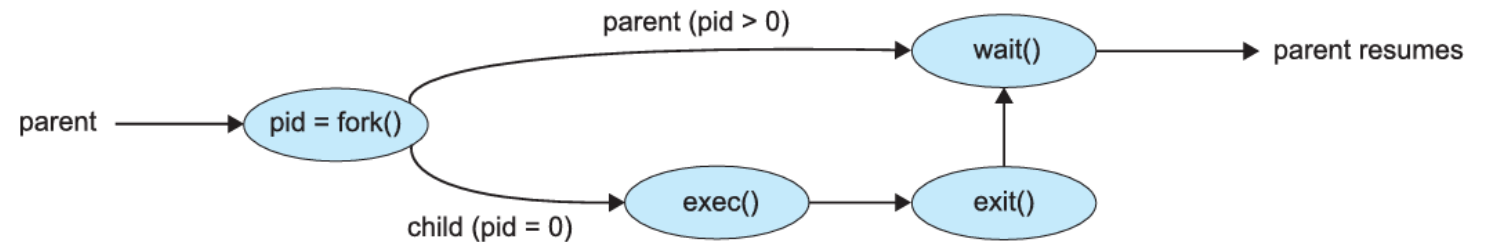
    if (fork() == 0) {
        // Child process
        printf("Hello from Child\n");
    } else {
        // Parent process
        printf("Hello from Parent\n");
    }

    return 0;
}
```

Compile & Run (Linux)

```
</> Bash

gcc program.c -o program
./program
```





Why does it create a child?



Because:

- Parent may need to continue running
- Parent may manage multiple child processes
- Parent may wait for child's result

Example:

- Shell creates child to run ls
- Shell continues running after ls finishes

Simple Real-Life Flow Example

When we type nano file.txt

Shell → fork()

Child process → exec("nano")

Nano runs

When nano exits → child terminates

Shell continues

Fork

fork() is a Linux system call for creating new processes

Used internally by shells, servers, and system daemons

Not visible to users directly

Powers multitasking in operating systems

Why fork() is Used

Process creation

Process isolation

Parallel execution

Multi-client handling

Backend Applications Using fork()

Shells (bash, zsh) → Execute commands

Web servers → Handle multiple client requests

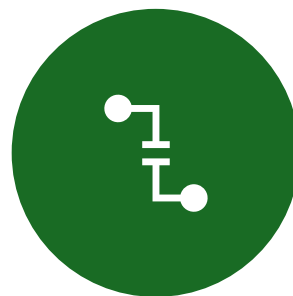
SSH servers → Manage remote sessions

Database servers → Create worker processes

Why It Is Powerful



Each process has its own memory space



Fault isolation (one crash does not break others)



Supports parallel request handling



Efficient for multi-user systems



Special Process Types



Zombie Process – Process has finished execution but parent has not read its exit status (wait() not called).

Orphan Process – Parent terminates before the child; child is adopted by init/systemd.

Interprocess Communication

Independent Process

- Does not share data with other processes
- Execution does not affect or depend on other processes

Cooperating Process

- Shares data with other processes
- Can affect or be affected by other processes
- Communication is required for coordination

Reasons for IPC



Information Sharing

Multiple applications may need access to the same data

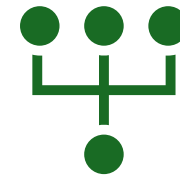
Supports concurrent access (e.g., copy-paste, shared resources)



Computation Speedup

A task is divided into subtasks executed in parallel

Speedup is possible only on multi-core systems



Modularity

System functions are divided into separate processes/threads

Improves structure, maintainability, and organization of the system

IPC Mechanisms



1. Shared Memory Model

A common memory region is created for processes

Processes communicate by reading and writing to shared memory

Faster communication

Requires synchronization to avoid data conflicts



2. Message Passing Model

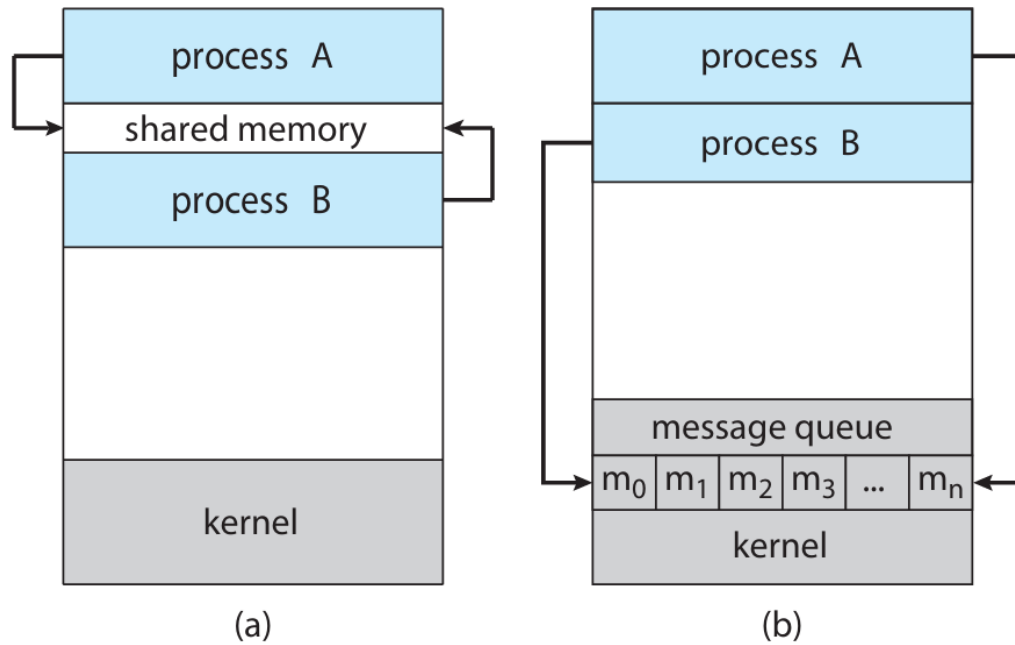
Processes communicate by sending and receiving messages

Data is exchanged through defined communication channels

Easier to implement and safer

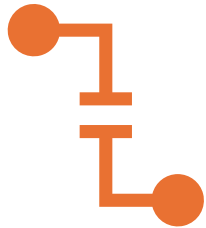
Slightly slower than shared memory

Illustration



- Shared Memory → Direct data access
- Message Passing → Data exchange via messages

Comparison: Shared Memory vs Message Passing



Message Passing

Suitable for exchanging small amounts of data

No shared data → No conflict issues

Easier to implement in distributed systems

Typically involves system calls → Slightly slower



Shared Memory

Faster for large data exchange

System call needed only to create shared region

After setup, communication happens via normal memory access

No kernel intervention during data exchange

Summary



A process is a running program



Each process has states and PCB



OS manages processes using scheduling



Context switching enables multitasking

IPC in Shared-Memory Systems



One process creates the shared memory region



Other processes attach it to their address space



Normally, OS prevents cross-memory access



Processes must agree to share access



Data structure and location are decided by processes



Processes must coordinate access to avoid conflicts

Producer–Consumer Problem (Cooperating Processes)

A common model of cooperating processes

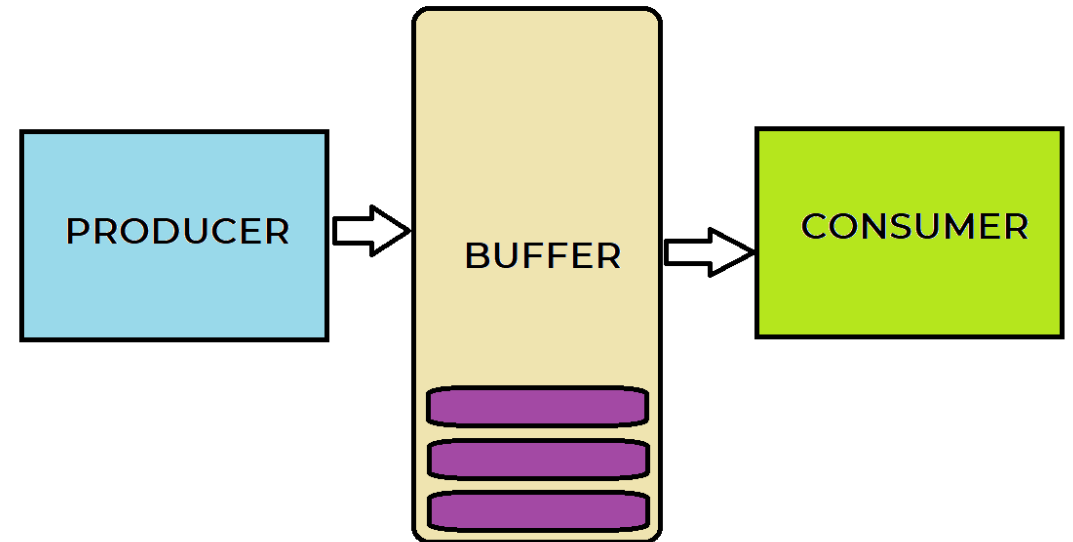
Producer generates data

Consumer uses that data

Example: Compiler → Assembler → Loader

Also models the **Client–Server paradigm**

Server provides resources; Client consumes them



Producer–Consumer Using Shared Memory

Shared memory is used as a common communication buffer

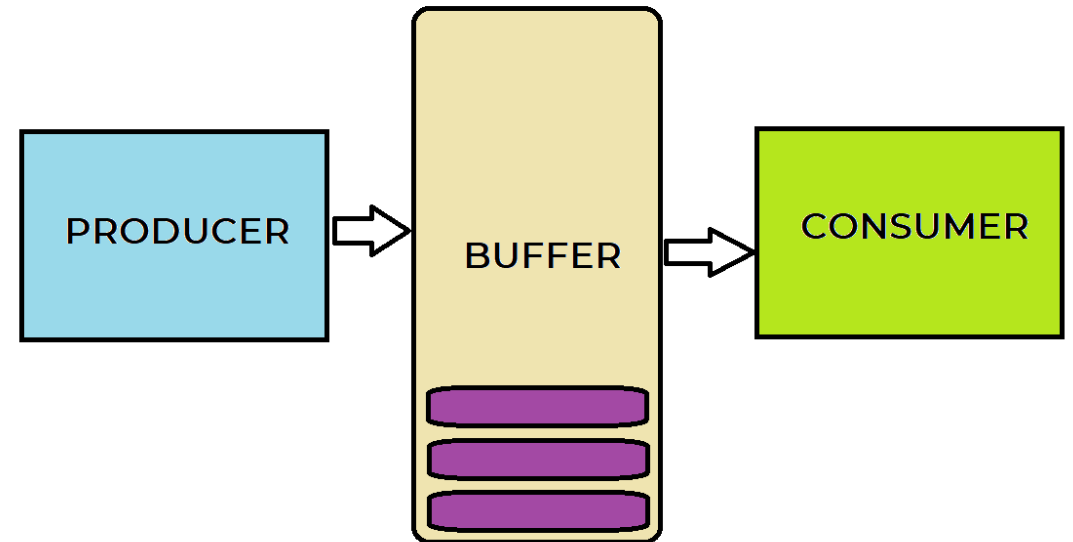
A buffer stores items produced and consumed

Producer adds items to the buffer

Consumer removes items from the buffer

Synchronization is required to avoid consuming unproduced items

Synchronization is **explicit and mandatory**



Synchronization techniques in shared memory



Locks

Mutex

Semaphore

Types of Buffers in Producer-Consumer

Unbounded Buffer

- No fixed size limit
- Producer never waits
- Consumer waits if buffer is empty

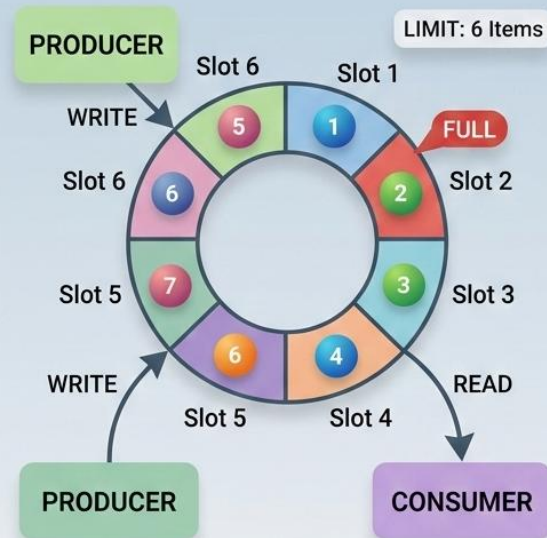
Bounded Buffer

- Fixed buffer size
- Producer waits if buffer is full
- Consumer waits if buffer is empty

BOUNDED BUFFER (IPC)

! BUFFER FULL!

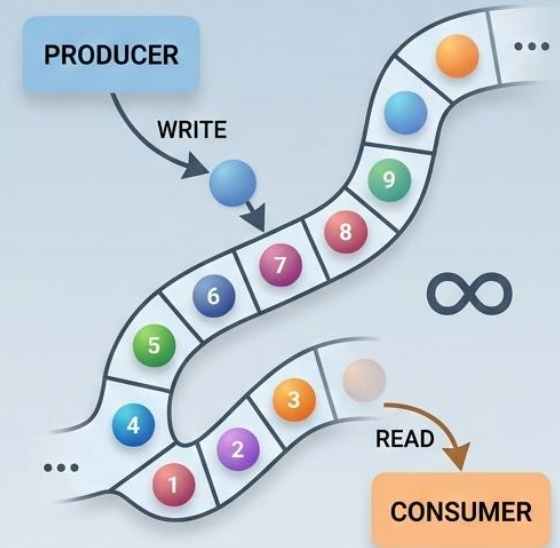
Producer must wait if all slots are filled



UNBOUNDED BUFFER (IPC)

✓ ALWAYS READY!

Producer never waits to write. Buffer can grow to any size.



Bounded vs Unbounded Buffer

Unbounded Buffer

- **Pros:**

- No fixed size limit
- Producer rarely blocks

- **Cons:**

- Can consume unlimited memory
- Risk of memory overflow

Bounded Buffer

- **Pros:**

- Controlled memory usage
- More practical for real systems

- **Cons:**

- Producer/consumer may wait
- Requires stronger synchronization

IPC in Message-Passing Systems

-
- Processes exchange data through messages
 - Synchronization is **implicit and built-in**.
 - Suitable for distributed systems (different machines over a network)
 - Example:** An Internet chat application where users communicate by sending and receiving messages.

IPC in Message-Passing Systems

- Provides two operations: send(message) and receive(message)
- Messages can be **fixed-size** or **variable-size**
- **Fixed-size Messages**
 - Simple system-level implementation
 - More difficult for programmers
 - **Example:** Network Packets in Operating Systems
- **Variable-size Messages**
 - Complex system implementation
 - Easier and more flexible for programming
 - **Example:** Pipes or Message Queues in Inter-Process Communication (IPC)

IPC in Message Passing Systems

-
- Communicating processes must have a **communication link**
 - Processes send and receive messages through this link
 - **Methods to create a link:**
 - Direct communication
 - Indirect communication

Direct Communication

Processes explicitly name each other

Communication uses:

- `send(P, message)`
- `receive(Q, message)`

Automatic link created between process pairs

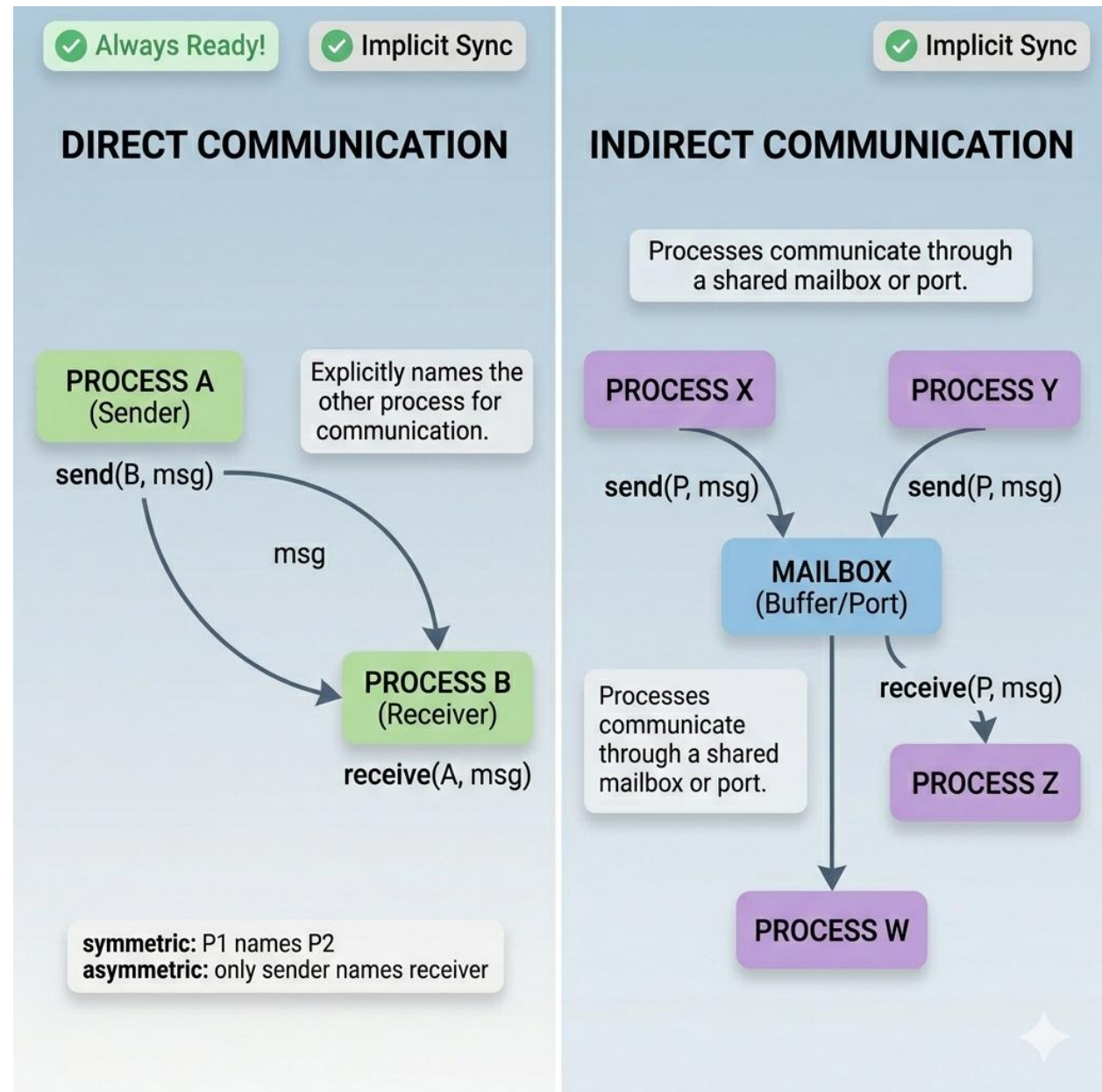
One unique link per pair of processes

Indirect Communication

- Processes communicate through **mailboxes/ports**
- Messages are sent to and received from a mailbox
- Link exists between process and mailbox
- Communication uses:
 - `send(A, message)`—Send a message to mailbox A.
 - `receive(A, message)`—Receive a message from mailbox A.
- More flexible and scalable than direct communication



Direct vs Indirect Communication



Direct Communication

- **Pros:**
 - Simple and direct process-to-process communication
 - Low overhead
 - Easy to implement
- **Cons:**
 - Less flexible
 - Not scalable for large systems
 - Tight coupling between processes
- **Example:**
 - Two programs communicating directly via socket connection
 - Client and server communicating with explicit addressing

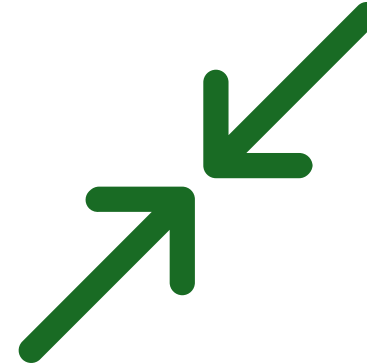
Indirect communication

- **Pros:**
 - Flexible and scalable
 - Communication through mailboxes/ports
 - Suitable for complex systems
- **Cons:**
 - Requires mailbox management
 - Slightly higher overhead
- **Example:**
 - Email system (messages sent to a mailbox)
 - Messaging apps using a server as an intermediate broker

Synchronization in IPC Message Passing



Blocking = Synchronous.



Non-blocking = Asynchronous.

Blocking (Synchronous)



Blocking Send: Sender waits until message is received



Blocking Receive: Receiver waits until message is available



Application: When reliable, ordered communication is required (e.g., request-response systems)



Limitation: Can reduce performance due to waiting

Non-Blocking (Asynchronous)

- **Non-Blocking Send:** Sender continues immediately after sending
- **Non-Blocking Receive:** Receiver checks and continues (message or null)
- **Application:**
 - High-performance or real-time systems
- **Limitation:**
 - More complex logic required
 - Must handle cases when message is not yet available



Buffering Types in Message Passing



Zero Capacity (No Buffer)

No message queue

Sender must wait until receiver is ready

Application:

- Strict synchronization (rendezvous communication), e.g. Phone call, Client Server Request (Database Query)

Limitation:

- Both processes must be ready at the same time
- Can reduce performance

Bounded Capacity

Fixed-size message queue

Sender waits if buffer is full

Application:

- Most real operating systems
- Producer–Consumer systems

Limitation:

- Requires synchronization
- Possible waiting if full/empty

Unbounded Capacity

Unlimited message queue size

Sender never waits

Application:

- Systems where message loss/blocking must be avoided

Limitation:

- Can consume excessive memory
- Risk of system overload

Examples of IPC Systems

- Different operating systems provide **different IPC mechanisms**.
- Common IPC systems include:
 - POSIX Shared Memory
 - Mach Message Passing
 - Windows IPC
 - Pipes (UNIX)

POSIX Shared Memory



Standard IPC method in **POSIX-compliant systems (Unix/Linux)**.



Processes share a **common memory region**.



Allows **very fast communication**.



Requires **synchronization mechanisms** to avoid conflicts.

Mach Message Passing



Used in the **Mach operating system architecture**.



Communication occurs through **sending and receiving messages**.



Uses **mailboxes (ports)** for message exchange.



Provides **flexible and secure process communication**.

Windows IPC

Windows provides several mechanisms for **communication between processes**.

IPC allows processes to **exchange data and coordinate tasks**.

Common Windows IPC Methods

- **Named Pipes** – Communication between related or unrelated processes.
- **Shared Memory (Memory-Mapped Files)** – Processes share the same memory region for fast data exchange.
- **Message Passing** – Processes send messages to each other through the OS.

Pipes (UNIX IPC)

- One of the **earliest IPC mechanisms in UNIX systems**.
- Allows **data flow between processes**.
- Typically used for **communication between parent and child processes**.
- Two types:
 - **Ordinary Pipes**
 - **Named Pipes (FIFOs)**

Ordinary Pipes



Ordinary pipes are a simple IPC mechanism used in operating systems.



Allow **two processes to communicate** with each other.

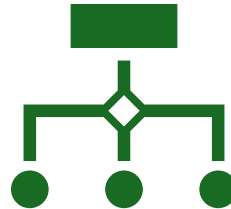


Commonly used in **UNIX and Windows systems**.

Producer–Consumer Model



Communication follows the **producer–consumer pattern**.



Producer process → writes data to the pipe.



Consumer process → reads data from the pipe.

Unidirectional Communication

Ordinary pipes support **one-way communication only**.

Data flows from **write end** → **read end**.

If **two-way communication** is needed, **two pipes** must be created.

Pipe Creation in UNIX

- Created using the system call:
 - `pipe(int fd[])`
- The file descriptors represent pipe ends:
 - `fd[0]` → **Read end** of the pipe
 - `fd[1]` → **Write end** of the pipe

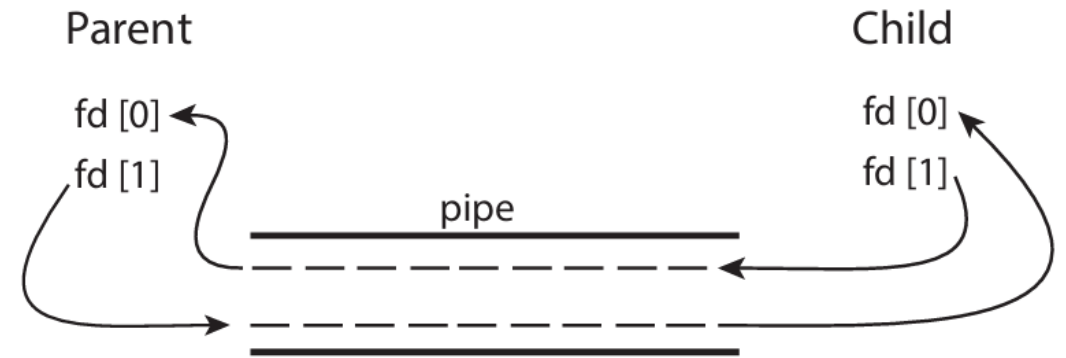


Figure 3.20 File descriptors for an ordinary pipe.



Named Pipes (FIFOs)

- Pipes that have a **name in the file system**.
- **No parent-child relationship required** between processes.
- Support **bidirectional communication**.
- Allow communication between **unrelated processes**.

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

-- OPERATOR CLASSES --

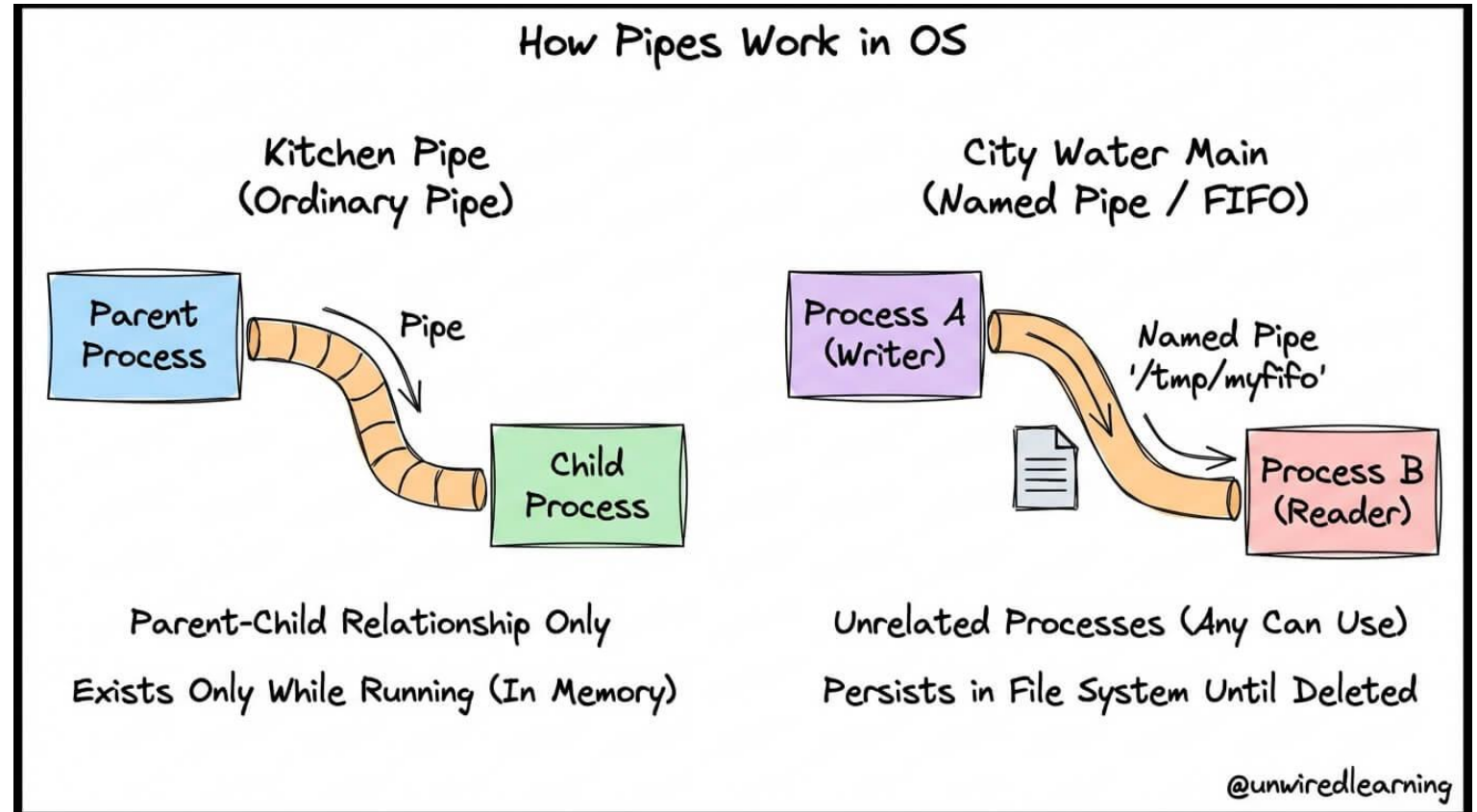
```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```

Named Pipes

- Created using commands like **mkfifo** (in UNIX/Linux).
- Persist in the system **until deleted**.
- Can be accessed like a **regular file**.
- Supported by **both UNIX and Windows systems** (implementation differs).

Illustration



Anonymous vs Named Pipes

Feature	Anonymous Pipes	Named Pipes
Process Relationship	Related processes	Unrelated processes
File System Presence	No	Yes
Lifetime	Temporary	Exists until deleted
Creation	pipe()	mkfifo